

Dynamic meta-programming, Domain-specific languages and LLM-backed semantic primitives



Václav Pech

NPRG014 2026/2027

<http://www.vaclavpech.eu>

@vaclav_pech

Homework reflections

- Shared view
 - Teach the coding skills (without AI)
 - Focus on high-level principles more
 - Dedicated lectures on AI coding processes

Homework reflections

- Some of the unique ideas
 - Learning through suffering (monkey brains)
 - Accelerate learning with AI tutors
 - Test on understanding AI generated code
 - Collect typical AI mistakes
 - AI can formulate prompts better
 - Communication, collaboration, understanding

Agenda

- Dynamic dispatch and meta-programming
- LLM-based semantic primitives
- Builders
- Domain specific languages

Part 4

Dynamic meta-programming

Topics

Dynamic dispatch

Dynamic cast

Dynamic object creation

Categories

Meta-programming

Method call dispatch

Static - the target method is decided at compile-time

Dynamic - the target method is decided at run-time

Method call dispatch

Object-oriented languages typically support **dynamic dispatch** based on the **receiver's runtime type** - virtual methods implemented using a virtual method table

Method overloading is typically resolved statically based on the declared types of the receiver and arguments.

Static dispatch

At compile-time using the compile-time type of the arguments

```
def calculate(String value)
```

```
def calculate(Integer value)
```

```
def calculate(Object value)
```

```
Object v = 10;
```

```
calculate(v); //Which method to call?
```

Dynamic dispatch

At run-time using the run-time type of the arguments

def calculate(String value)

def calculate(Integer value)

def calculate(Object value)

Object v = 10;

calculate(v); //Which method to call?

Dynamic object creation

Duck-typing

```
Calculator c = [ add : {a, b → a + b},  
                multiply : {a, b → a * b},  
                increment : {it + 1}  
              ] as Calculator
```

```
assert 6 == c.multiply(2, 3)
```

Type coercion

Implicit

Runnable r = {println 'Asynchronous'}

Integer i = '42'

Explicit

def f = '3.14' as Float

def s = [1, 2, 3] as Set

Type coercion

Defined using naming convention

- *asType*, *asBoolean*, *isCase*

Category (extension methods)

```
StringUtils.countMatches(myString, 'Groovy')
```



```
use(StringUtils) {  
    myString.countMatches('Groovy')  
}
```

Category (extension methods)

Attach a method to an existing class

Kotlin:

```
fun String.countMatches(String match) {...}
```

Groovy:

```
class Util {  
    static String countMatches(target, match) {  
    }  
}
```

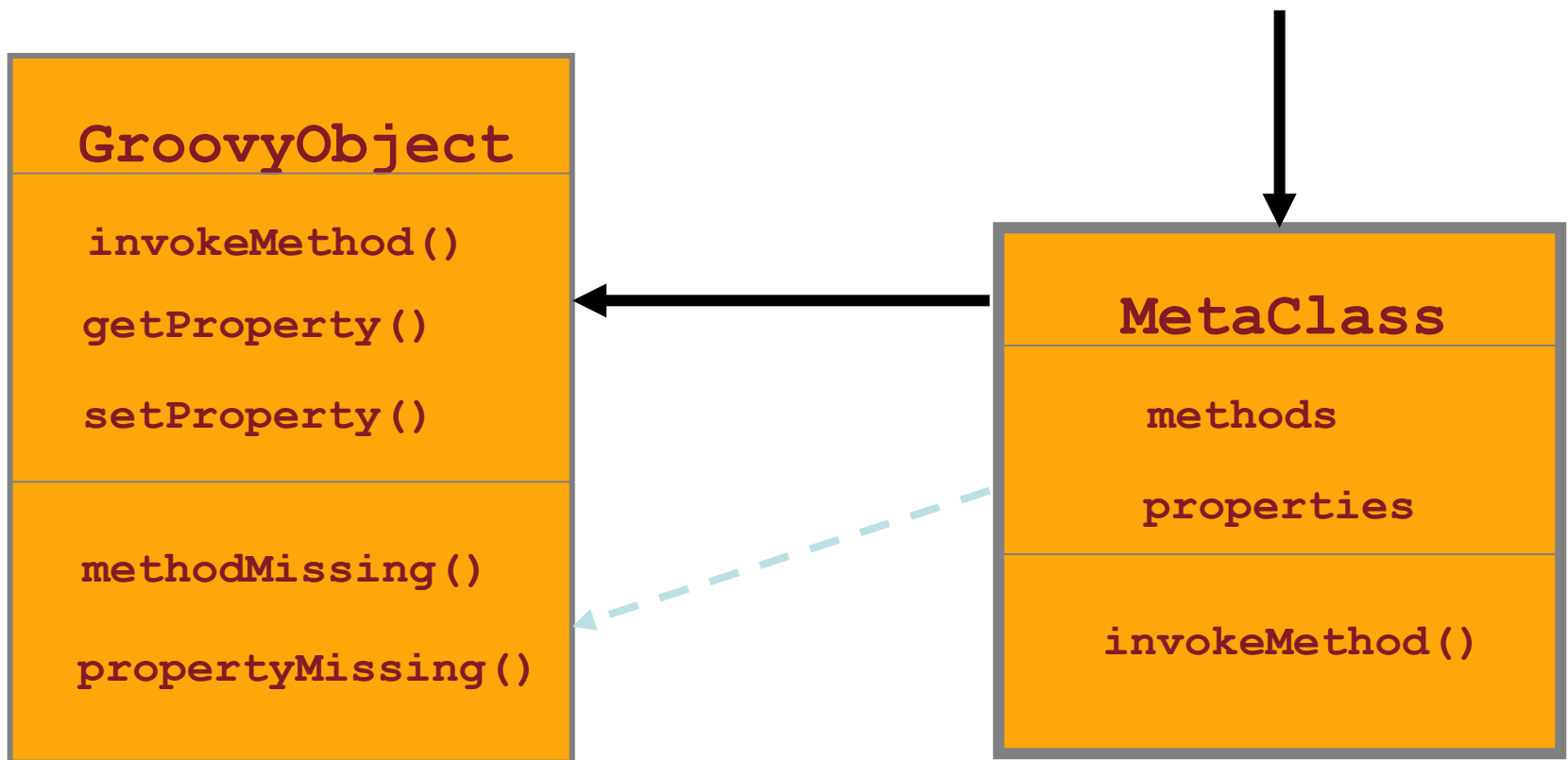
Open classes in Groovy

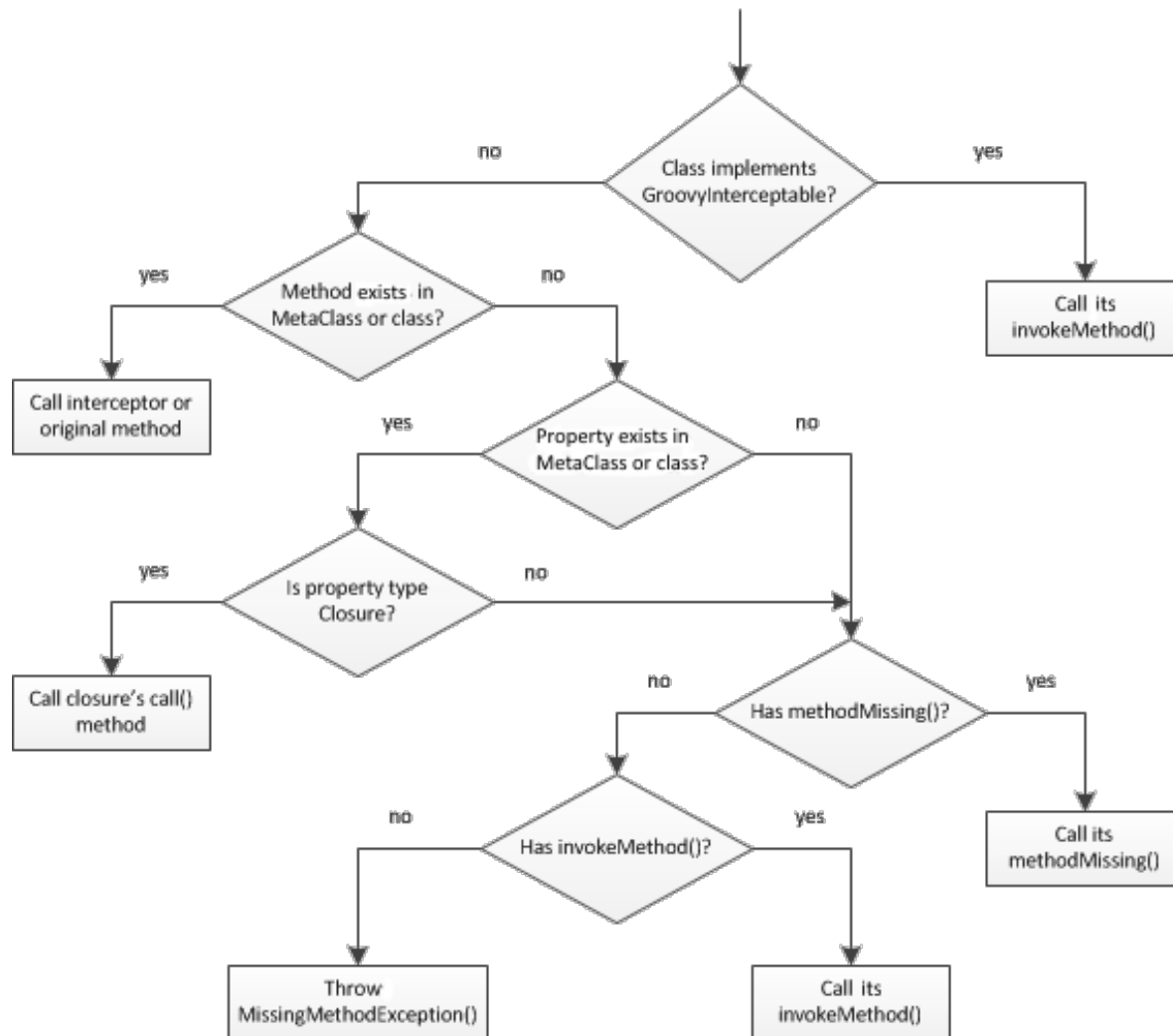
Problem: It must work with Java classes

How to add methods to statically compiled Java classes?

- Add a Map-like property named *metaClass* to every class or object
- Use it to store additional methods/properties
- Dispatch method calls dynamically

Dynamic method invocation





Meta class

```
String.metaClass.backToFront = {->  
    delegate[-1..0]  
}  
  
println 'cimanyd si yvoorG'.backToFront()
```

Meta class

```
String.metaClass.backToFront = {->  
    delegate[-1..0]  
}
```

```
println 'cimanyd si yvoorG'.backToFront()
```

```
String myText = "..."
```

```
myText.metaClass.backToFront = {->  
    delegate[-1..0]  
}
```

```
println 'cimanyd si yvoorG'.backToFront()
```

Querying objects' methods

`o.respondsTo()`

`o.hasProperty()`

`o.metaClass.getMetaMethod(name, args)`

`o.metaClass.getMetaProperty(name)`

invokeMethod

```
public Object invokeMethod(String methodName, Object o) {  
    def matcher = methodName =~ 'findAll(.*?)Employees'  
    if (matcher.matches()) {  
        String action = matcher.group(1)  
        if (action.toLowerCase() in ['senior', 'experienced', 'expert']) {  
            return employees.findAll {it.age > 30}  
        }  
        if (action.toLowerCase() in ['junior', 'fresh']) {  
            return employees.findAll {it.age <= 30}  
        }  
    }  
  
    return [];  
}
```

Missing handlers

```
def methodMissing(String name, args) {  
    println "$name ${args.join(' and ')}"  
    println "${args.join(' and ')} say $name back"  
    println ""  
}  
  
def propertyMissing(String name) { name }
```

Part 5

LLM-backed semantic primitives

Examples

```
if (text.feelsLike(„happy“) {
```

```
    ...
```

```
}
```

```
while (observation.contradicts(theory) {
```

```
    ...
```

```
}
```

LLM-based primitives

- Transformation (*String* $\rightarrow$ *String*)
 - *summarize, spellCorrect, rephrase, shorten, ...*
- Classification (*String* $\rightarrow$ *Enum* | *Boolean*)
 - *sentimentOf, topicOf, ...*
 - *funny(), supports, contradicts, isCodingRequest, ...*
- Similarity (*String* $\rightarrow$ *Double* | *Boolean*)
 - *similarTo, talksAbout, feelsLike, ...*

LLM-based primitives

- **Transformation** ($String \rightarrow String$)
 - *summarize, spellCorrect, rephrase, shorten, ...*
- **Classification** ($String \rightarrow Enum \mid Boolean$)
 - *sentimentOf, topicOf, ...*
 - *funny(), supports, contradicts, isCodingRequest, ...*
- **Similarity** ($String \rightarrow Double \mid Boolean$)
 - *similarTo, talksAbout, feelsLike, ...*

Transformation String → String

```
String spellCorrect(String text) {  
    def llm = new LLMChatConnector()  
    llm.systemPrompt = '''  
        When given text, return a version with all spelling errors corrected.  
        You must not alter or reduce the meaning of the text.  
    ...  
    def shortText = llm.chat(text)  
    shortText  
}
```

```
String.metaClass.spellCorrect {->  
    spellCorrect(delegate)  
}
```

```
println ('Long stori shot, I came laet to the porty..'.spellCorrect())
```

LLM-based primitives

- Transformation (*String* $\rightarrow$ *String*)
 - *summarize, spellCorrect, rephrase, shorten, ...*
- **Classification** (*String* $\rightarrow$ *Enum* | *Boolean*)
 - *sentimentOf, topicOf, ...*
 - *funny(), supports, contradicts, isCodingRequest, ...*
- Similarity (*String* $\rightarrow$ *Double* | *Boolean*)
 - *similarTo, talksAbout, feelsLike, ...*

Classification String → Enum

```
String sentimentOf(String text) {  
    llm = new LLMChatConnector()  
    llm.systemPrompt = '''  
You are a sentiment classifier.  
Classify the supplied text using exactly one of these labels:  
positive  
negative  
neutral  
Return only the label and nothing else.  
'''.trim()  
  
    String result = llm.chat(text).trim().toLowerCase()  
    return result  
}
```

Classification String → Boolean

```
boolean isCodingRequest(String text) {  
    llm = new LLMChatConnector()  
    llm.systemPrompt = '''  
You are a text request detector that identifies requests for writing code.  
Decide whether the supplied text represents a request for code creation.  
Reply either 'YES' or 'NO'.  
Return only the single word and nothing else.  
'''.trim()  
  
    def result = llm.chat(text).trim().toUpperCase()  
    switch (result) {  
        case 'YES': return true  
        case 'NO': return false  
    }  
    return false  
}  
  
String.metaClass.isCodingRequest = {->  
    isCodingRequest(delegate)  
}
```

Decisions API

Traditional LLM API

INPUT

"Analyze this bug report and determine its severity."

OUTPUT

"This appears to be a high-severity bug because it affects..."

The application must interpret or parse generated content.

Decisions API

INPUT

"Select severity: LOW, MEDIUM, HIGH, CRITICAL."

OUTPUT

✓ **HIGH**

Probability: 0.87 (illustrative)

The result is already structured for programmatic use.

Decisions API

Newly proposed by OpenAI

Allows fast **classification**:

- Tool selection
- Result verification
- Quality assesment
- Risk classification
- etc.

Decisions example

```
def request = [  
  model: 'nimble',  
  state: text,  
  questions: [  
    coding: [  
      type: 'noul',  
      instructions: '''  
        Does the text request the creation or modification  
        of source code? Answer true for requests to write,  
        generate, implement, or modify code.  
        Answer false for explanations, questions about  
        programming, or other non-coding requests.  
      '''.trim()  
    ]  
  ]  
]
```

ollama pull nimble

Decisions example

```
def request = [  
  model: 'tev1:0.8b',  
  state: text,  
  questions: [  
    coding: [  
      type: 'noul',  
      instructions: '''  
        Does the text request the creation or modification  
        of source code? Answer true for requests to write,  
        generate, implement, or modify code.  
        Answer false for explanations, questions about  
        programming, or other non-coding requests.  
      '''.trim()  
    ]  
  ]  
]
```

ollama pull tev1:0.8b

LLM-based primitives

- Transformation ($String \rightarrow String$)
 - *summarize, spellCorrect, rephrase, shorten, ...*
- Classification ($String \rightarrow Enum \mid Boolean$)
 - *sentimentOf, topicOf, ...*
 - *funny, supports, contradicts, isCodingRequest, ...*
- **Similarity** ($String \rightarrow Double \mid Boolean$)
 - *similarTo, talksAbout, feelsLike, ...*

Motivation

Question: Recommend a place to visit based on my notes in this folder.

1. Find notes that are related to travelling
2. Process the relevant notes with LLM

Motivation

Question: Recommend a place to visit based on my notes in this folder.

1. Find notes that are **related** to travelling
 - A similarity filter, do not feed LLM with all notes
2. Process the **relevant** notes with LLM

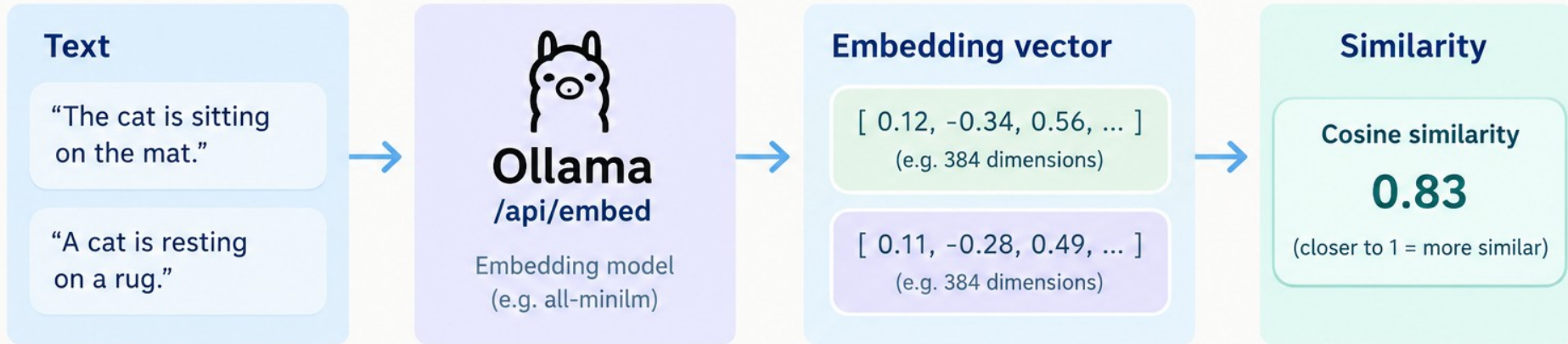
Embeddings

Embedding – a vector in a multi-dimensional vector space

Similarity – distance of their embedding vectors

Embeddings with Ollama

Turn text into vectors. Compare meaning, not just words.



Example embeddings (shortened)

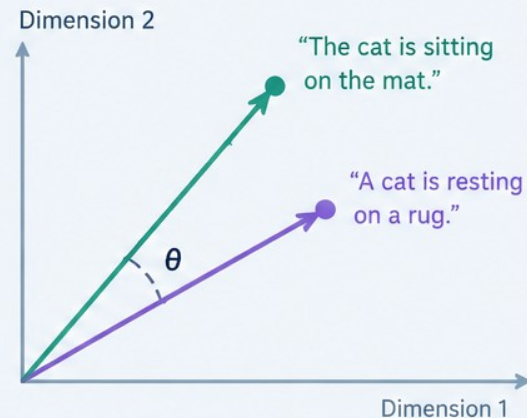
“The cat is sitting on the mat.” → [0.12, -0.34, 0.56, -0.21, 0.08, 0.33, ...]

“A cat is resting on a rug.” → [0.11, -0.28, 0.49, -0.18, 0.07, 0.31, ...]

Similarity (cosine)

$$\cos(\theta) = \frac{v_1 \cdot v_2}{\|v_1\| \|v_2\|} \rightarrow \mathbf{0.83}$$

In the embedding space (2D view)



Semantically similar texts end up close to each other in the embedding space.

Embedding models

Generating text and **representing** text as a semantic vector are different tasks

Embedding models tend to be smaller and faster

- Example models: *all-minilm*, *nomic-embed-text*

Embeddings

- *embedding()* = semantic representation
- *cosineSimilarity()* = mathematical comparison
- *similarity()* = convenient combined operation

```
double similarity(String text1, String text2) {  
    List<Double> v1 = embedding(text1)  
    List<Double> v2 = embedding(text2)  
  
    cosineSimilarity(v1, v2)  
}
```

```

double similarity(String text1, String text2) {

    List<Double> v1 = embedding(text1)
    List<Double> v2 = embedding(text2)

    cosineSimilarity(v1, v2)
}

```

```

List<Double> embedding(String text) {

    def requestBody = [
        model: model,
        input: text
    ]

    def response = post(requestBody)

    def embeddings = response?.embeddings

    // /api/embed returns a list of embeddings because it also supports
    // embedding multiple input strings in one request.
    def vector = embeddings[0]

    vector.collect { it as Double }
}

```

```

double similarity(String text1, String text2) {
    List<Double> v1 = embedding(text1)
    List<Double> v2 = embedding(text2)

    cosineSimilarity(v1, v2)

    String baseUrl = 'http://localhost:11434/api/embed'

    // Small and lightweight embedding model.
    String model = 'all-minilm'
}

```

```

List<Double> embedding(String text) {
    def requestBody = [
        model: model,
        input: text
    ]

    def response = post(requestBody)

    def embeddings = response?.embeddings

    // /api/embed returns a list of embeddings because it also supports
    // embedding multiple input strings in one request.
    def vector = embeddings[0]

    vector.collect { it as Double }
}

```

LLM-based primitives

- Transformation ($String \rightarrow String$)
 - *summarize, spellCorrect, rephrase, shorten, ...*
- Classification ($String \rightarrow Enum \mid Boolean$)
 - *sentimentOf, topicOf, ...*
 - *funny, supports, contradicts, isCodingRequest, ...*
- Similarity ($String \rightarrow Double \mid Boolean$)
 - ***similarTo, talksAbout, feelsLike, ...***

similarTo

```
boolean similarTo(String text1, String text2) {  
    embeddings.similarity(text1, text2) >= similarityThreshold  
}
```

talksAbout

```
boolean talksAbout(String document, String topic) {  
    if (!document || !topic) {  
        return false  
    }  
  
    def chunks = embeddings.splitIntoChunks(document)  
  
    chunks.any { chunk ->  
        embeddings.similarity(chunk, topic) >= similarityThreshold  
    }  
}
```

feelsLike

```
private static final Map<String, String> FEELING_PROTOTYPES = [  
    enthusiastic:  
        'enthusiastic, excited, energetic, strongly positive and eager',  
  
    disappointed:  
        'disappointed, dissatisfied, let down, negative reaction',  
  
    angry:  
        'angry, irritated, hostile, frustrated and upset',  
  
    calm:  
        'calm, peaceful, relaxed and emotionally composed',  
  
    anxious:  
        'anxious, worried, nervous, fearful and uneasy',  
  
    humorous:  
        'humorous, playful, funny and intended to amuse'  
]
```

```
boolean feelsLike(String text, String feeling) {  
    String prototype = FEELING_PROTOTYPES[feeling.toLowerCase()]  
    embeddings.similarity(text, prototype) >= similarityThreshold  
}
```

Limitations of embeddings

- Struggle to resolve
 - Contradictions
 - Logical operators and negation
 - Nuanced sentiment, sarcasm, etc.
- Often ***pre-process/filter*** texts before LLMs

Part 6

Domain Specific Languages

BDD - Spock

```
class DataDriven extends Specification {  
    def "maximum of two numbers"() {  
        expect:  
        Math.max(a, b) == c  
        where:  
        a | b | c  
        7 | 3 | 7  
        4 | 5 | 5  
        9 | 9 | 9  
    }  
}
```

Gradle

```
apply plugin: 'kotlin-android'

apply plugin: 'kotlin-android-extensions'

android {
    compileSdkVersion 28
    defaultConfig {
        applicationId "com.example.myapplication"
        minSdkVersion 16
        targetSdkVersion 28
        versionCode 1
        versionName "1.0"
        testInstrumentationRunner "androidx.test.runner.AndroidJUnitRunner"
    }
    buildTypes {
        release {
            minifyEnabled false
            proguardFiles getDefaultProguardFile('proguard-android-optimize.txt'), 'proguard-rules.pro'
        }
    }
}

dependencies {
    implementation fileTree(dir: 'libs', include: ['*.jar'])
    implementation "org.jetbrains.kotlin:kotlin-stdlib-jdk7:$kotlin_version"
    implementation 'androidx.appcompat:appcompat:1.0.2'
    implementation 'androidx.core:core-ktx:1.0.2'
    implementation 'androidx.constraintlayout:constraintlayout:1.1.3'
    implementation 'com.google.android.material:material:1.0.0'
    testImplementation 'junit:junit:4.12'
    androidTestImplementation 'androidx.test:runner:1.1.1'
    androidTestImplementation 'androidx.test.espresso:espresso-core:3.1.1'
}
```

DSL

- Limited purpose language
- Targeted to a particular domain
- Friendlier API to a framework
 - External
 - SQL, HTML, CSS, ...
 - Internal

DSL – Date manipulation

```
use (org.codehaus.groovy.runtime.TimeCategory) {  
    println "Tomorrow: ${1.day.from.today}"  
    println "A week ago: ${1.week.ago}"  
    println "Date: ${1.month.ago + 1.week + 2.hours - 5.minutes}"  
    println "Date ${ (1.month + 10.days).ago}"  
}
```

DSL – Hibernate criteria

```
def participants = Participant.createCriteria().list {  
    gt('age', age)  
    or{  
        eq('interest', 'Java')  
        eq('interest', 'Groovy')  
    }  
    jug {  
        ilike('country', 'de')  
    }  
    order('lastName', 'asc')  
}
```

DSL – Account manipulation

```
Money money = new Money(amount: 350, currency: 'eur')  
getAccount('Account1').withdraw money  
getAccount('Account3').deposit money
```



```
"Account1" >> 350.eur >> "Account3"
```

Why DSLs

Domain expressiveness – code closely matches domain concepts, improving readability.

Reduced boilerplate – conciseness compared to general-purpose language constructs.

Improved communication – shared language between developers and domain experts.

Higher abstraction – hides low-level details, focusing on "what" not "how."

Consistency & correctness – enforces domain-specific rules directly in syntax.

Rapid prototyping – easier experimentation with domain models and solutions.

Integration leverage – benefits from host language tooling (IDE, debuggers, libraries).

Easier maintenance – domain logic is explicit, reducing cognitive load over time.

Example – file access DSL

```
File.metaClass.div = { path ->
    new File(delegate, path)
}
```

Example – file access DSL

```
File.metaClass.div = { path ->  
    new File(delegate, path)  
}
```

```
def file = new File(".")/'test'/'hello'/'file.txt'
```

order cake with plums and apples
and cream to "Malostranske namesti"

```
order(cake) .with(plums) .and(apples)  
 .and(cream) .to("Malostranske namesti")
```

Builders

Construct hierarchies

- html, xml, json, swing, configuration
- objects
- db queries
- ...

Builders - GAnt

```
ant.sequential {  
    myDir = "target/AntTest/"  
    mkdir(dir: myDir)  
    copy(todir: myDir) {  
        fileset(dir: "src/test") {  
            include(name: "**/*.groovy")  
        }  
    }  
    List dirs = ['core', 'lib', 'engine', 'gui', 'db']  
    for(String currentDir:dirs) {  
        String targetDir="target/$currentDir"  
        mkdir(dir:targetDir)
```

Builders – Spring config

```
dataSource(BasicDataSource) {  
    driverClassName = "org.hsqldb.jdbcDriver"  
    url = "jdbc:hsqldb:mem:shopDB"  
}  
  
sessionFactory(ConfigurableLocalSessionFactoryBean) {  
    dataSource = dataSource  
    hibernateProperties = ["hibernate.hbm2ddl.auto": "create-drop",  
        "hibernate.show_sql": true]  
}  
  
calculator(demo.shop.CalculatorImpl) {bean ->  
    bean.singleton = true  
    bean.autowire = 'byType'  
}
```

JSON Builder

```
def builder = new JsonBuilder()
builder {
  model "gpt-5"
  messages([
    [role: "system", content: "You are a helpful assistant."],
    [role: "user", content: "Explain JsonBuilder in Groovy."]
  ])
  max_tokens 200
  temperature 0.7
}
```

Summary



Dynamic method call dispatch

Dynamic object creation

Categories

LLM-backed semantic primitives

Dynamic DSLs

Builders

References

<http://groovy-lang.org>

<https://martinfowler.com/books/dsl.html>

<https://ollama.com/>

<https://docs.langchain4j.dev/>

<https://www.jetbrains.com/koog/>